

3 Architecture and Design

3.1 Overall Architecture

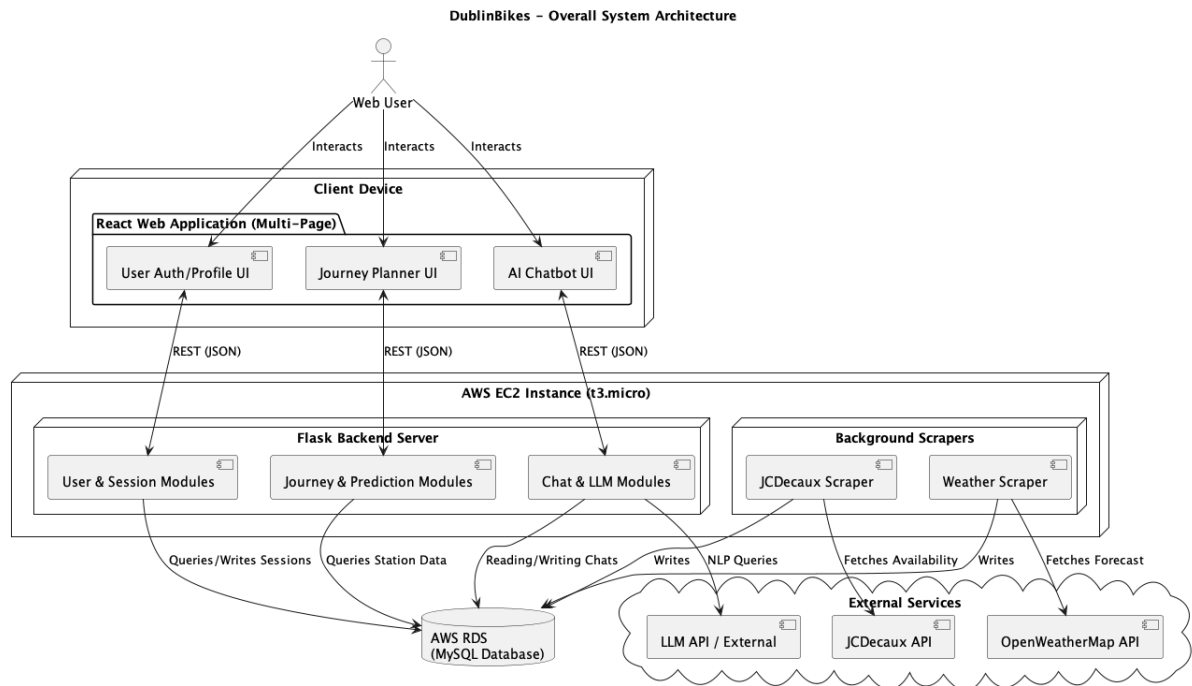


Figure 1 Overall System Architecture

Our project utilizes a decoupled client-server and cloud architecture, separating the front-end user interfaces, back-end web routing, and the data acquisition processes.

The architecture (Figure 1) illustrates a scalable Web application split into three robust functions: user authentication, journey planning, and an AI chatbot.

Client Device: A React-based multi-page application (MPA) utilising distinct route pages (/login, /maps, /chat). It communicates to the backend asynchronously via JSON REST APIs seamlessly across the layout.

Server Environment (AWS EC2): A scalable AWS EC2 micro instance hosts our core Python layer.

Flask Backend Server: Acts as the traffic controller, routing API requests, formatting data, and calling our predictive models.

Background Scrapers: Standalone scripts continuously gather live data from the JCDecaux API and OpenWeatherMap API, pushing raw data into our database independently of the Web Server.

Database Layer (AWS RDS): A managed, centralised AWS RDS MySQL instance that securely maintains static station references, dynamic availability histories, and weather data independent of the EC2 lifecycle.

This separation of concerns guarantees that long-running data-fetching tasks don't block API responses, and ensures that the client remains highly responsive.

3.2 Class Diagram

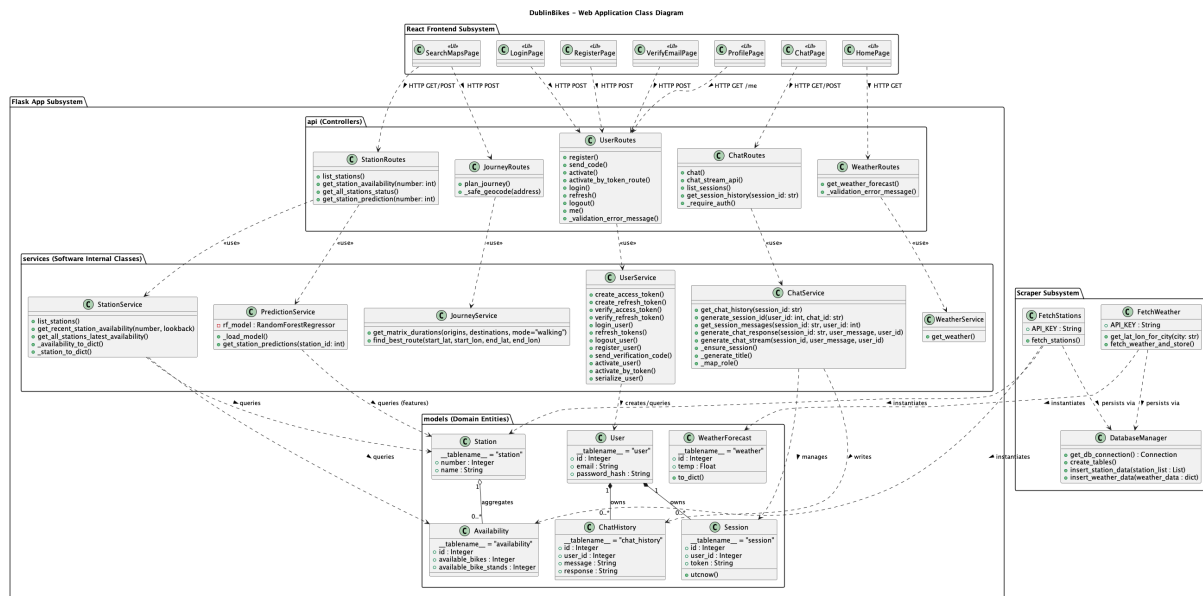


Figure 2 Web Application Class Diagram

The detailed Class Diagram (Figure 2) abstracts our underlying code into proper object-oriented software internal classes and domain entities.

Scraper Subsystem: Exhibits a collection mapping to core ingestion processes. It uses “FetchStations” and “FetchWeather” logic which delegate raw formatting to an internal “DatabaseManager” to persist data instances.

Flask App Subsystem: Illustrates our service-oriented architecture covering the three main functions.

Models (Domain Entities): Database-bound objects mapping our RDS tables, including our core domain (“Station”, “Availability”, “WeatherForecast”), alongside user and AI chat state logic (“User”, “Session”, “ChatHistory”).

Services (/services): Business logic implementations explicitly separated: “UserService” (authentication), “JourneyService” (route predictions), and “ChatService” (message processing).

Routes (/api): The Flask routing layer intercepts specific domain requests (“UserRoutes”, “JourneyRoutes”, “ChatRoutes”) and forwards the payload dictionaries into the respective services without holding heavy calculation logic itself.

React Frontend Subsystem: Outlines our front-end functional UI components (“LoginPage”, “SearchMapsPage”, “ChatPage”) and how they trigger specific “HTTP POST/GET” requests to their corresponding API branches in the Flask-App schema.

3.3 Sequence Diagrams of Core Use Cases

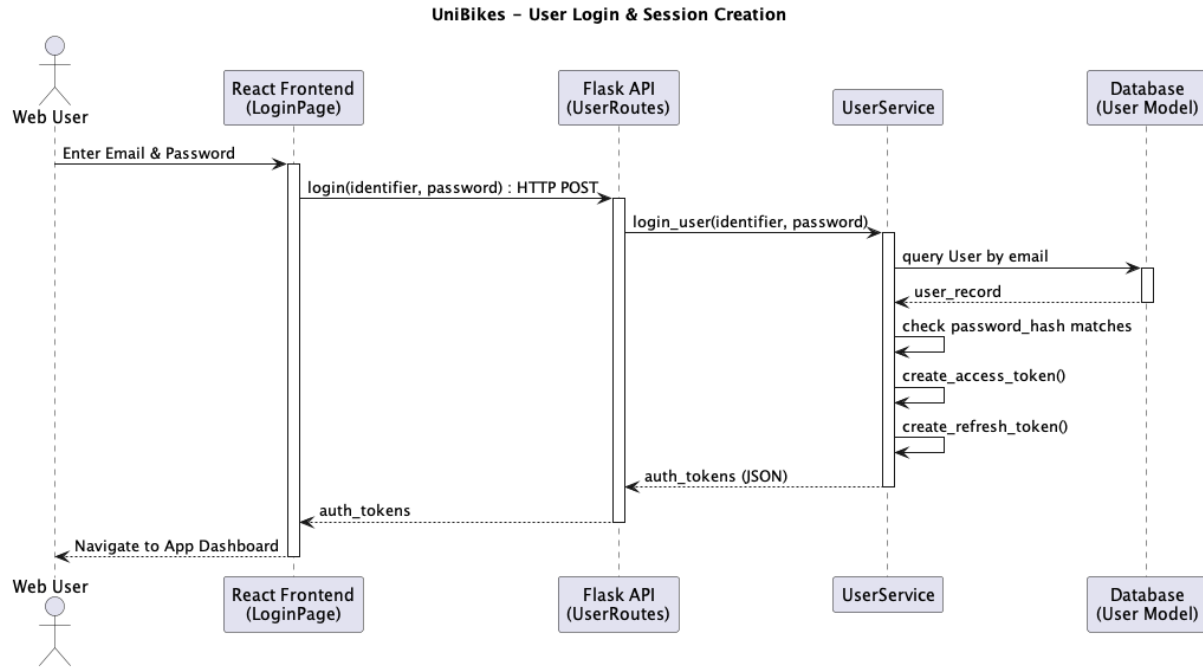


Figure 3 User Login and Session Creation

User Login (Figure 3): Details the flow from the React “LoginPage” to the generation of JWT auth keys in the “UserService”, concluding with the issuance of stateless JWT access and refresh tokens back to the client interface.

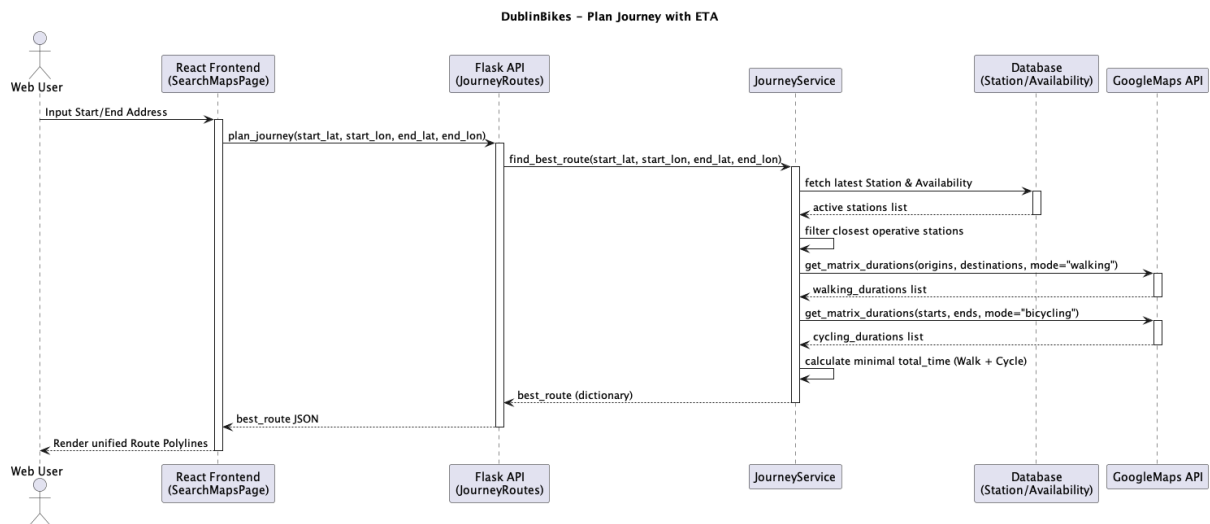


Figure 4 Journey Planner

Journey Planner (Figure 4): Initiated from “SearchMapsPage”, the “JourneyService” utilises matrix computations directly against the GoogleMaps API (“get_matrix_durations()”), combining optimal cycling and walking computations geographically.

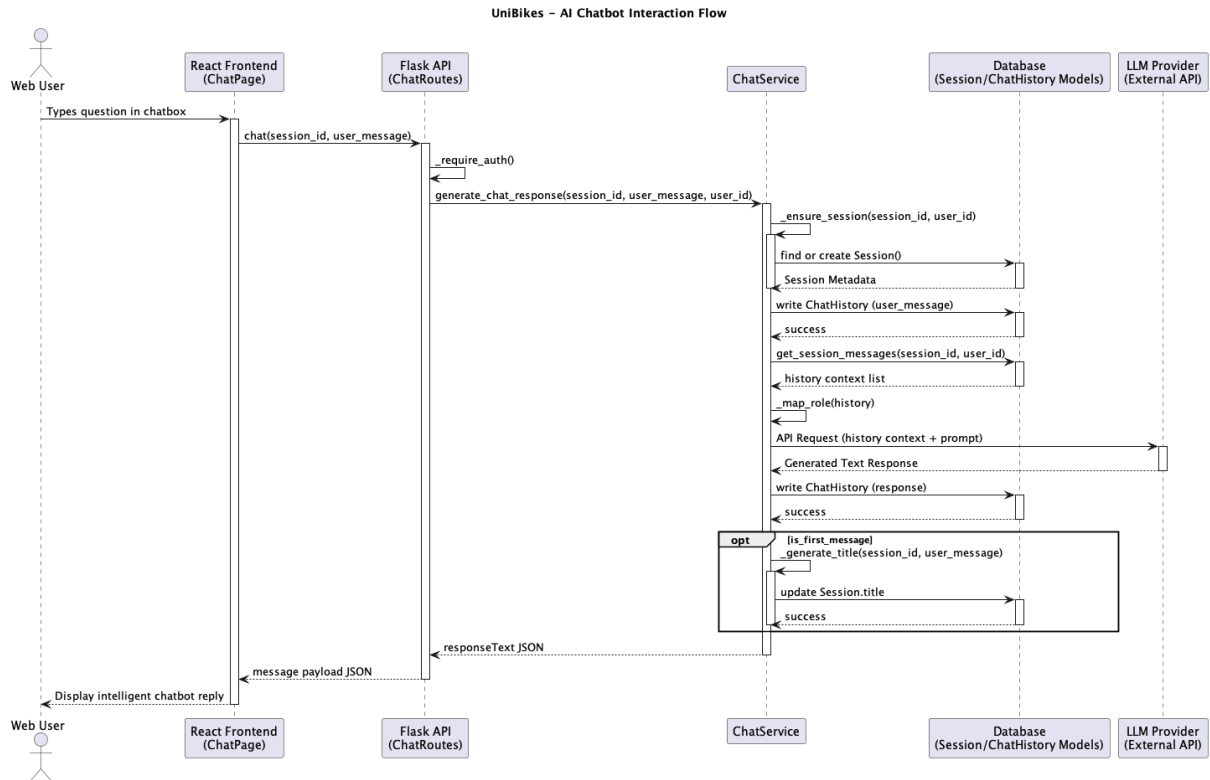


Figure 5 AI Chatbot

AI Chatbot (Figure 5): The message passes structural auth checks (“_require_auth()”) and persists domain state via “ensure_session()”, referencing the “Session” metadata table. The “ChatService” fetches interaction context (“ChatHistory”) prior to transmitting prompts to the external LLM provider, while dynamically utilising the LLM (“_generate_title()”) to auto-classify a chat topic title on initial interaction.

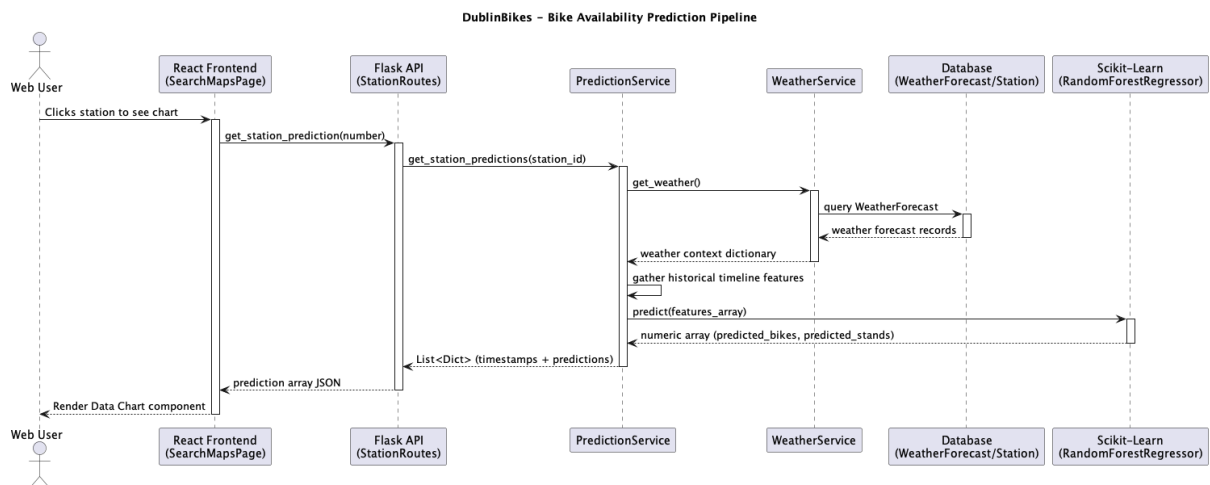


Figure 6 Bike Availability Prediction

Bike Availability Prediction (Figure 6): The request flows from “StationRoutes” down to the “PredictionService”, appending contextual features mapped from “WeatherService” against the local “Database” cache before engaging the predictive memory-bound “RandomForestRegressor”. To

address typical initial cold-start latencies inherent to ML modeling, the “PredictionModel” referenced in the prediction sequence is pre-loaded directly into Flask's memory upon initialization.

3.4 Database Design and Design Choices

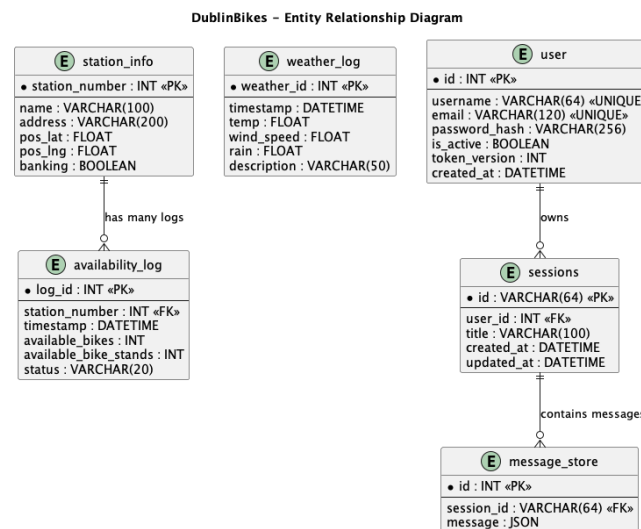


Figure 7 Entity Relationship

Managed AWS RDS (Relational Cloud SQL): We opted for a cloud-managed Relational Database (AWS RDS for MySQL) rather than a local SQLite file or NoSQL. Shared bike systems and meteorological data are highly structured datasets. Given that the Machine Learning modeling pipeline requires performing time-series transformations and heavy “JOIN” operations across station IDs and timestamps, a robust cloud SQL environment was naturally the correct fit.

Data normalization: Static factors like a station's latitude, longitude, and internal naming (“station_info”) remain constant. Instead of redundantly saving these details with every availability check, they are separated. The “availability_log” records highly volatile factors (every 5-10 minutes) leveraging Foreign Keys mapping back to the static tables, reducing storage footprint and read redundancy.

Indexing: Because querying time-aligned weather logs and bike availability histories spans millions of rows, we introduced specific database indexing on the “timestamp” and “station_number” columns. This design drastically diminished query delays when compiling features to refresh our predictive Random Forest classifiers.

Stateful LLM persistence & auth isolation: To securely expand the application beyond public bike metrics, we isolated authentication and Chatbot states via the “user”, “sessions”, and “message_store” tables. The “user” records enforce robust identity and JWT validations, mapped via foreign key explicitly to Chatbot metadata (“sessions”). To accommodate the vast, unpredictable text structure characteristic of Large Language Models via LangChain, the “message_store” table purposefully archives deep ongoing chat interactions leveraging native JSON columns.

3.5 Additional Material

All original UML files are securely tracked in our GitHub repository
https://github.com/ucdsc/docs/tree/main/UML_files.